

CCF 全国信息学奥林匹克联赛（NOIP2020）复赛

提高组模拟

时间：2020 年 12 月 02 日 8:00–12:00

一. 题目概况

中文题目名称	球员	显示器	牧羊人	画图
英文题目与子目录名	player.cpp	display.cpp	shepherd.cpp	paint.cpp
可执行文件名	传统型	传统型	传统型	传统型
输入文件名	player	display	shepherd	paint
输出文件名	player	display	shepherd	paint
每个测试点时限	2秒	4秒	1秒	3秒
测试点数目	3	6	4	4
每个测试点分值	捆绑测试	捆绑测试	捆绑测试	捆绑测试
附加样例文件				
结果比较方式	全文比较（过滤行末空格及文末回车）			
题目类型	传统	传统	传统	传统
运行内存限制	512MB	512MB	512MB	512MB

二. 提交源程序文件名

对于C++语言	player.cpp	display.cpp	shepherd.cpp	paint.cpp
对于C语言	player.c	display.c	shepherd.c	paint.c
对于pascal语言	player.pas	display.pas	shepherd.pas	paint.pas

三. 编译命令（不包含任何优化开关）

对于C++语言	g++ -o player player.cpp -lm	g++ -o display display.cpp -lm	g++ -o shepherd shepherd.cpp -lm	g++ -o paint paint.cpp -lm
对于C语言	gcc -o player player.c -lm	gcc -o display display.c -lm	gcc -o shepherd shepherd.c -lm	gcc -o paint paint.c -lm
对于pascal语言	fpc player.pas	fpc display.pas	fpc shepherd.pas	fpc paint.pas

注意事项：

- 1、文件名（程序名和输入输出文件名）必须使用英文小写。
- 2、C/C++中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
- 3、全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz，内存 32GB。上述时限以此配置为准。
- 4、只提供 Linux 格式附加样例文件。
- 5、特别提醒：评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以其为准。

1. 球员

(player.cpp/c/pas)

【题目描述】

老师们已经知道学生喜欢睡觉，Soaring 是这项记录保持者。他只会吃饭或玩 FIFA20 时才会醒来。因此，他经常做关于足球的梦，在他最近的一次梦中，他发现自己成了皇家马德里足球俱乐部的总经理。

他的工作是挑选 N 名球员争取在下个赛季打败巴塞罗那队，但是董事会有两个特殊的要求。具体如下：

①所有运动员姓氏的长度必须不同。

②每个运动员的姓氏必须是长度比其长的所有其他运动员姓氏的连续子串

为了让工作变得简单，Soaring 将潜在的球员分成 N 类，第 i 类的球员的姓氏恰好有 i 个字母，且每一类恰好有 K 个球员。

Soaring 想知道有多少种不同的方法选出满足要求的 N 个球员。答案对 (10^9+7) 取余。

【输入格式】

输入文件名为 player.in。

第一行包含两个整数 N ($1 \leq N \leq 50$) 和 K ($1 \leq K \leq 1500$)。

接下来 N 行，第 i 行包含 K 个球员的姓氏，每个姓氏都是由 i 个英语小写字母组成，注意同一类的 K 个球员姓氏不一定不同。

【输出格式】

输出文件名为 player.out。

输出答案。

【输入输出样例 1】

player.in	player.out
3 2 a b ab bd abc abd	5

【输入输出样例 2】

player.in	player.out
3 3 a b c aa ab ac aaa aab aca	6

【输入输出样例 3】

player.in	player.out
-----------	------------

3 1 a a bc def	0
-------------------------	---

【数据范围】

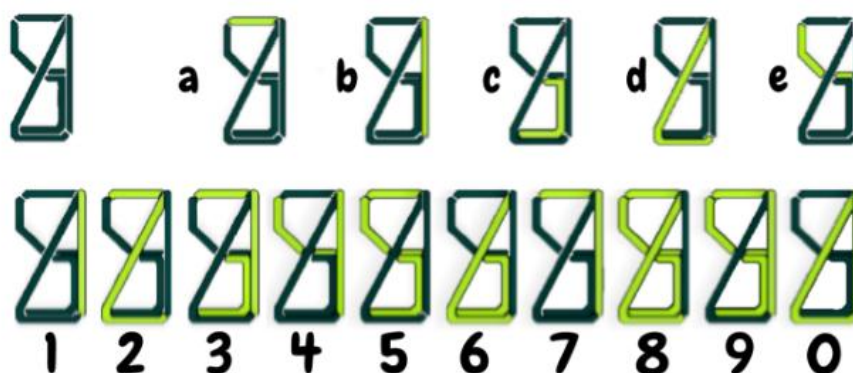
子任务	限制	分值
1	$N=5$ 且 $K=10$	22
2	$N=50$ 且 $K=100$	33
3	无任何其他限制	45

2. 显示器

(display.cpp/c/pas)

【题目描述】

你可能很熟悉的 7 段显示器，它被广泛用于各种设备上显示数字，如手表或计算器。由于其简洁、直观及美观，这种设计已被世界各国所接受。尽管如此，年轻的 Soaring 反对 7 段式设计，并声称仅使用 5 段显示器（下图中 a, b, c, d, e 这 5 段）就可以有效地获得同样的功能。



5-segment display design – segments are labeled with letters from a to e.

Soaring 决定在其最强的运动项目足球比赛中进行推广使用。他的革命性设计将被用在下一轮奥运会预选赛中。他目前正在说服足协董事会。

球员替换牌在足球比赛中用以显示被换下球员的球衣号码，Soaring 设计的替换牌有 M 个 5 段显示器，从左到右依次排开。Soaring 在游说时，先把替换牌设置成数 X ，同时，然后接下来每秒进行一次操作。操作有两种：

- ①打开一个当前关闭的段
- ②关闭一个当前打开的段

Soaring 一共进行 N 次操作，并且保证每进行 K 次操作后，替换牌上都会显示一个有效的数，即牌上的每一位显示器上显示的数字都是正确的。此外，只要正确显示前导 0，少于 M 位的球衣号码也认为是有效的。

Soaring 感兴趣的是，对于每个最终状态（0 到 $10^M - 1$ 之间的整数），他有多少种不同的操作方法来实现。当然，他需要遵守前面提到的操作要求。我们认为只要某一刻替换牌显示的状态不一样，我们就认为这两种操作序列不同。

由于答案可能非常大，所以只需要输出模 $(10^9 + 7)$ 即可。

【输入格式】

输入文件名为 display.in。

第一行那包含整数 M, N, K ($1 \leq K \leq N$) 和 X ($0 \leq X < 10^M$)

【输出格式】

输出文件名为 display.out。

第 i 行输出替换牌经过 N 次操作后最终显示数 $i-1$ 的方法数，答案模 (10^9+7) 。

【输入输出样例 1】

display. in	display. out
1 2 1 5	0
	0
	0
	1
	0
	2
	0
	0
	0
	0

【输入输出样例 2】

display. in	display. out
1 3 3 8	0
	0
	0
	6
	0
	13
	0
	0
	0
	0

【输入输出样例 3】

display. in	display. out
1 4 2 4	24
	0
	8
	0
	37
	0
	4
	28
	4

	24
--	----

【数据范围】

子任务	限制	分值
1	$M=1, 1 \leq N \leq 12$	6
2	$M=1, 1 \leq N \leq 10^{15}$	15
3	$M=2, 1 \leq N \leq 1500, K = N$	12
4	$M=2, 1 \leq N \leq 10^{15}, 1 \leq K \leq 15$	12
5	$M=2, 1 \leq N \leq 10^{15}, 1 \leq K \leq 1500$	15
6	$M=2, 1 \leq N \leq 10^{15}$	40

3. 牧羊人

(shepherd. cpp/c/pas)

【题目描述】

一棵树上有 K 只羊，树是一个没有环的简单连通图。这棵树有 N 个节点，用 1 到 N 表示。树的每个节点最多只能住一只羊。Soaring 意识到，狼迟早会学会爬树。

为了保护羊，Soaring 决定安排一些牧羊人住在树中某些节点上，以确保每只羊至少被一个牧羊人保护。众所周知，每个牧羊人都会保护所有离他最近的羊，且只会保护它们。羊和牧羊人的距离等于两者之间简单路径的边数包含绵羊的节点和包含 shepherd 的节点之间唯一路径上的节点。此外，牧羊人可以跟羊住在一起，当然，这种情况下，牧羊人只保护这只羊。

计算最少需要放置多少个牧羊人，使得每只羊都至少被一个牧羊人保护。另外，输出一种放置方案。

【输入格式】

输入文件名为 shepherd. in。

第一行两个整数 N 和 K ($1 \leq K \leq N$)

接下来 $N-1$ 行，每行包含两个整数 a_i 和 b_i ($1 \leq a_i, b_i \leq N$)，描述树中的边。

接下来一行包含 K 个不同的整数 o_i ($1 \leq o_i \leq N$)，表示 K 只羊居住的节点编号。

【输出格式】

输出文件名为 shepherd. out。

第一行输出一个数字 X ，表示最小需要的牧羊人数。

第二行输出 X 个空格分隔的整数，表示牧羊人居住的节点编号。

如果有多个方案，输出其中任何一个。

【输入输出样例 1】

shepherd. in	shepherd. out
4 2	2
1 2	1 3
2 3	
3 4	
1 4	

【输入输出样例 2】

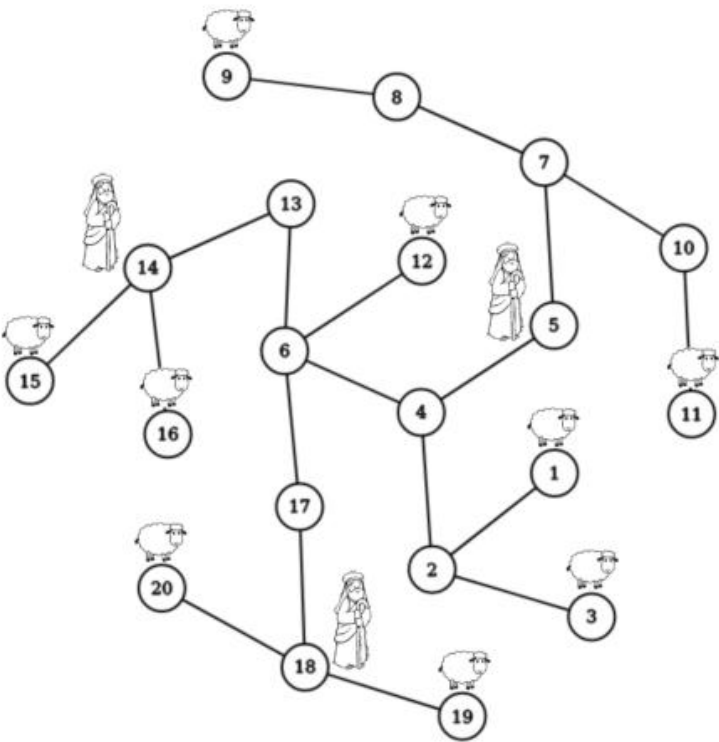
shepherd. in	shepherd. out
9 5	3
1 2	1 4 9
2 3	
3 4	
3 5	

1 6	
1 7	
7 8	
8 9	
2 5 6 7 9	

【输入输出样例 3】

shepherd. in	shepherd. out
20 9	3
1 2	5 14 18
2 3	
2 4	
4 5	
4 6	
5 7	
7 8	
8 9	
7 10	
10 11	
6 12	
6 13	
6 17	
13 14	
14 15	
14 16	
17 18	
18 19	
18 20	
1 3 9 11 12 15 16 19 20	

【样例 3 解释】



【数据范围】

子任务	限制	分值
1	$1 \leq N \leq 500\ 000$ 每个节点 x 与 $x+1$ ($1 \leq x \leq N-1$) 相连	8
2	$1 \leq K \leq 15, 1 \leq N \leq 500\ 000$	18
3	$1 \leq N \leq 2\ 000$	23
4	$1 \leq N \leq 500\ 000$	51

4. 画图

(paint.cpp/c/pas)

【题目描述】

Soaring 用画图工具的画图区域视为 R 行 S 列的网格图。网格的每个格点可以看作是可以用于 10^5 种颜色来着色的单个像素点。

当 Soaring 用“桶”这个工具把颜色为 B 的像素点 (r, s) 着成颜色 A 时，那么所有颜色为 B 且与像素点 (R, S) 四连通(上, 下, 左, 右)的区域全部被着成颜色 A 。

给 Soaring 一个初始图像以及 Q 条指令， Q 指令依次执行。每条指令都告诉 Soaring 在哪个像素上用哪个颜色应用桶工具。

Soaring 需要输出最终的图像。

【输入格式】

输入文件名为 paint.in。

第一行包含整数 R 和 S 。

接下来 R 行，每行包含 S 个小于 100,000 的非负整数，表示初始图像每个像素点的颜色。准确地说，第 i 行第 j 个数字表示像素 (i, j) 的颜色。

接下来一行输入整数 Q 。

接下来 Q 行，其中第 i 行包含整数 $r_i, s_i, c_i (1 \leq r_i \leq R, 1 \leq s_i \leq S, 0 \leq c_i < 100\,000)$ ，描述第 i 条指令，表示要在像素 (r_i, s_i) 上使用颜色 c_i 进行“桶”工具操作。

【输出格式】

输出文件名为 paint.out。

以与输入相同的格式输出最终图像的状态。

【输入输出样例 1】

paint.in	paint.out
12 11	1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1	1 1 1 1 4 4 4 1 1 1 1 1
1 1 1 1 0 0 0 1 1 1 1 1	1 1 1 4 4 4 4 4 1 1 1 1
1 1 1 0 0 0 0 0 1 1 1 1	1 1 4 4 4 4 4 4 4 1 1 1
1 1 0 0 0 0 0 0 0 1 1 1	1 4 4 4 3 3 3 4 4 4 1 1
1 0 0 0 2 2 2 0 0 0 1 1	1 4 4 4 3 3 3 4 4 4 1 1
1 0 0 0 2 2 2 0 0 0 1 1	1 4 4 4 3 3 3 4 4 4 1 1

1 0 0 0 2 2 2 0 0 0 1	1 4 4 4 4 4 4 4 4 4 1
1 0 0 0 0 0 0 0 0 0 1	1 1 4 4 4 2 4 4 4 1 1
1 1 0 0 0 2 0 0 0 1 1	0 1 1 4 4 2 4 4 1 1 0
0 1 1 0 0 2 0 0 1 1 0	0 0 1 1 4 4 4 1 1 0 0
0 0 1 1 0 0 0 1 1 0 0	0 0 0 1 1 1 1 1 1 0 0 0
0 0 0 1 1 1 1 1 0 0 0	
2	
5 5 3	
6 2 4	

【输入输出样例 2】

paint. in	paint. out
4 4	1 1 1 3
1 0 1 3	1 1 2 2
1 3 2 2	1 1 1 2
3 3 1 2	3 3 1 3
2 2 1 3	
3	
1 2 3	
3 2 1	
4 2 3	

【输入输出样例 3】

paint. in	paint. out
6 6	1 3 1 2 2 2
1 2 1 2 2 2	3 1 3 1 3 1
3 1 2 1 3 1	3 3 3 3 3 3
3 3 2 3 2 2	3 3 1 3 3 3
2 3 1 3 3 2	3 3 3 3 3 3
3 3 3 3 3 3	3 3 3 3 3 1
2 3 2 2 2 1	

4	
6 2 2	
3 5 2	
3 2 3	
1 2 3	

【数据范围】

子任务	限制	分值
1	$1 \leq R \cdot S \leq 10\,000, 1 \leq q \leq 10\,000$	8
2	$R=1, 1 \leq S \leq 200\,000, 1 \leq Q \leq 100\,000$	9
3	$1 \leq R \cdot S \leq 200\,000, 1 \leq Q \leq 100\,000$ 每个时刻每个像素都只会被着成颜色 0 或 1	31
4	$1 \leq R \cdot S \leq 200\,000, 1 \leq Q \leq 100\,000$	52